

Step 1 — Setup & Load Dataset

```
!pip install datasets sacrebleu evaluate bert-score transformers sentencepiece
```

```
100.8/100.8 kB 9.1 MB/s eta 0:00:00  
84.1/84.1 kB 10.2 MB/s eta 0:00:00  
61.1/61.1 kB 6.8 MB/s eta 0:00:00
```

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
from datasets import load_dataset  
import warnings  
warnings.filterwarnings('ignore')  
  
print("✅ All libraries imported!")
```

```
✅ All libraries imported!
```

```
dataset = load_dataset("c4/filt/iitb-english-hindi")  
print(dataset)
```

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated req
README.md:      3.14k/? [00:00<00:00, 126kB/s]

dataset_infos.json: 100%                                953/953 [00:00<00:00, 35.9kB/s]

data/train-00000-of-                                190M/190M [00:02<00:00, 502MB/s]
00001.parquet: 100%

data/validation-00000-of-                            85.7k/85.7k [00:00<00:00, 301kB/s]
00001.parquet: 100%

data/test-00000-of-                                500k/500k [00:00<00:00, 1.82MB/s]
00001.parquet: 100%

Generating train split: 100%                        1659083/1659083 [00:03<00:00, 393547.22 examples/s]

Generating validation split: 100%                    520/520 [00:00<00:00, 23072.69 examples/s]

Generating test split: 100%                          2507/2507 [00:00<00:00, 127032.56 examples/s]

DatasetDict({
  train: Dataset({
    features: ['translation'],
    num_rows: 1659083
  })
  validation: Dataset({
    features: ['translation'],
    num_rows: 520
  })
})

```

```

# Take 50,000 pairs (full dataset is too large for Colab free tier)
train_data = dataset['train'].select(range(50000))
test_data = dataset['test'] # official test set (~2507 pairs)

print(f"Train size: {len(train_data)}")
print(f"Test size: {len(test_data)}")

# Peek at data
print("\nSample pair:")
print("EN:", train_data[0]['translation']['en'])
print("HI:", train_data[0]['translation']['hi'])

```

```

Train size: 50000
Test size: 2507

```

```

Sample pair:

```

EN: Give your application an accessibility workout
 HI: अपने अनुप्रयोग को पहुंचनीयता व्यायाम का लाभ दें

Step 2 — Preprocessing & Cleaning

```
!pip install indic-nlp-library -q
```

```

===== 40.3/40.3 kB 3.5 MB/s eta 0:00:00
===== 7.7/7.7 MB 74.7 MB/s eta 0:00:00
===== 121.1/121.1 kB 12.1 MB/s eta 0:00:00

```

```

import re
import unicodedata
from indicnlp.normalize.indic_normalize import IndicNormalizerFactory

print("✅ Preprocessing libraries ready!")

```

✅ Preprocessing libraries ready!

```

# English cleaner
def clean_english(text):
    text = text.strip()
    text = re.sub(r'http\S+', '', text) # remove URLs
    text = re.sub(r'^\w\s\.\,\\!\\?]', '', text) # remove special chars
    text = re.sub(r'\s+', ' ', text) # remove extra spaces
    return text.lower()

# Hindi cleaner
factory = IndicNormalizerFactory()
normalizer = factory.get_normalizer("hi")

def clean_hindi(text):
    text = text.strip()
    text = normalizer.normalize(text) # normalize Devanagari
    text = re.sub(r'^\u0900-\u097F\s\.\,\\!\\?]', '', text) # keep only Hindi c
    text = re.sub(r'\s+', ' ', text) # remove extra spaces
    return text

print("✅ Cleaning functions defined!")

```

✅ Cleaning functions defined!

```

# Clean train data
train_en = [clean_english(x['translation']['en']) for x in train_data]
train_hi = [clean_hindi(x['translation']['hi']) for x in train_data]

# Clean test data
test_en = [clean_english(x['translation']['en']) for x in test_data]

```

```
test_hi = [clean_hindi(x['translation']['hi']) for x in test_data]
```

```
print(f"✅ Train cleaned: {len(train_en)} pairs")
print(f"✅ Test cleaned: {len(test_en)} pairs")
```

```
✅ Train cleaned: 50000 pairs
✅ Test cleaned: 2507 pairs
```

```
print("=== BEFORE vs AFTER ===\n")
```

```
raw_en = train_data[0]['translation']['en']
raw_hi = train_data[0]['translation']['hi']
```

```
print(f"Raw EN:      {raw_en}")
print(f"Cleaned EN: {train_en[0]}")
print()
print(f"Raw HI:      {raw_hi}")
print(f"Cleaned HI: {train_hi[0]}")
```

```
=== BEFORE vs AFTER ===
```

```
Raw EN:      Give your application an accessibility workout
Cleaned EN:  give your application an accessibility workout
```

```
Raw HI:      अपने अनुप्रयोग को पहुंचनीयता व्यायाम का लाभ दें
Cleaned HI:  अपने अनुप्रयोग को पहुंचनीयता व्यायाम का लाभ दें
```

Step 3 — EDA (Exploratory Data Analysis)

Sentence Length Analysis

```
# Calculate lengths
train_en_lengths = [len(s.split()) for s in train_en]
train_hi_lengths = [len(s.split()) for s in train_hi]

print("=== English Sentence Stats ===")
print(f"Avg length:  {np.mean(train_en_lengths):.2f} words")
print(f"Max length:  {max(train_en_lengths)} words")
print(f"Min length:  {min(train_en_lengths)} words")

print("\n=== Hindi Sentence Stats ===")
print(f"Avg length:  {np.mean(train_hi_lengths):.2f} words")
print(f"Max length:  {max(train_hi_lengths)} words")
print(f"Min length:  {min(train_hi_lengths)} words")
```

```
=== English Sentence Stats ===
Avg length:  4.42 words
Max length:  115 words
Min length:  0 words
```

```

=== Hindi Sentence Stats ===
Avg length: 4.02 words
Max length: 113 words
Min length: 0 words

```

Plot Sentence Length Distribution

```

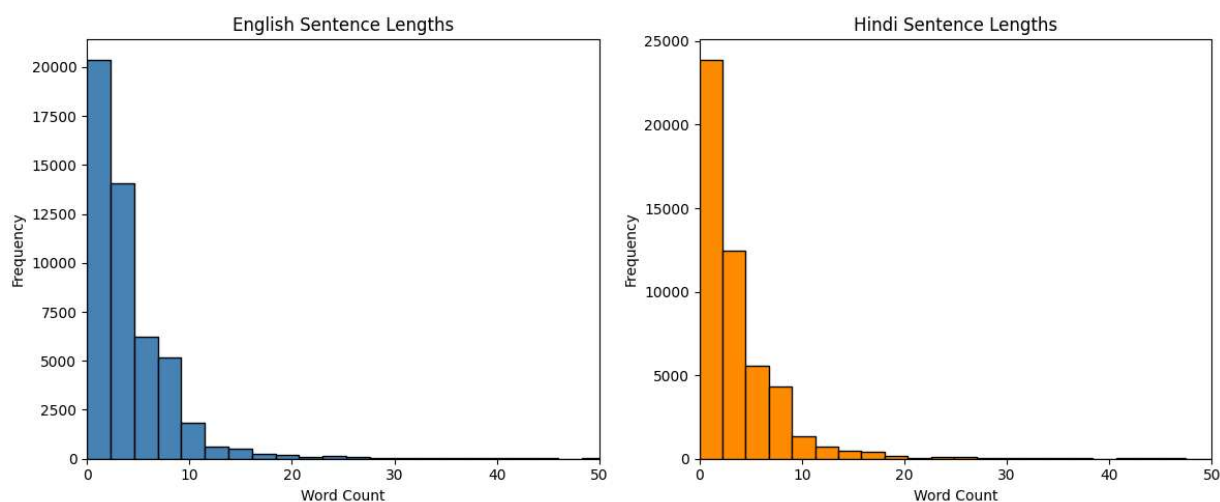
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.hist(train_en_lengths, bins=50, color='steelblue', edgecolor='black')
plt.title('English Sentence Lengths')
plt.xlabel('Word Count')
plt.ylabel('Frequency')
plt.xlim(0, 50)

plt.subplot(1, 2, 2)
plt.hist(train_hi_lengths, bins=50, color='darkorange', edgecolor='black')
plt.title('Hindi Sentence Lengths')
plt.xlabel('Word Count')
plt.ylabel('Frequency')
plt.xlim(0, 50)

plt.tight_layout()
plt.savefig('sentence_lengths.png', dpi=150)
plt.show()
print("✅ Plot saved!")

```



✅ Plot saved!

Vocabulary Size

```

from collections import Counter

# Build vocabularies
en_vocab = Counter(" ".join(train_en).split())

```

```
hi_vocab = Counter(" ".join(train_hi).split())
```

```
print(f"English Vocabulary Size: {len(en_vocab):,} unique words")
print(f"Hindi Vocabulary Size: {len(hi_vocab):,} unique words")
print(f"\nTop 10 English words: {en_vocab.most_common(10)}")
print(f"\nTop 10 Hindi words: {hi_vocab.most_common(10)}")
```

English Vocabulary Size: 4,459 unique words

Hindi Vocabulary Size: 4,226 unique words

Top 10 English words: [('the', 12754), ('to', 6310), ('_', 5536), ('of', 4838),

Top 10 Hindi words: [('के', 6300), ('को', 5523), ('करें', 4096), ('है', 4063), ('

Word Frequency Chart

```
# Top 20 words
top_en = en_vocab.most_common(20)
top_hi = hi_vocab.most_common(20)

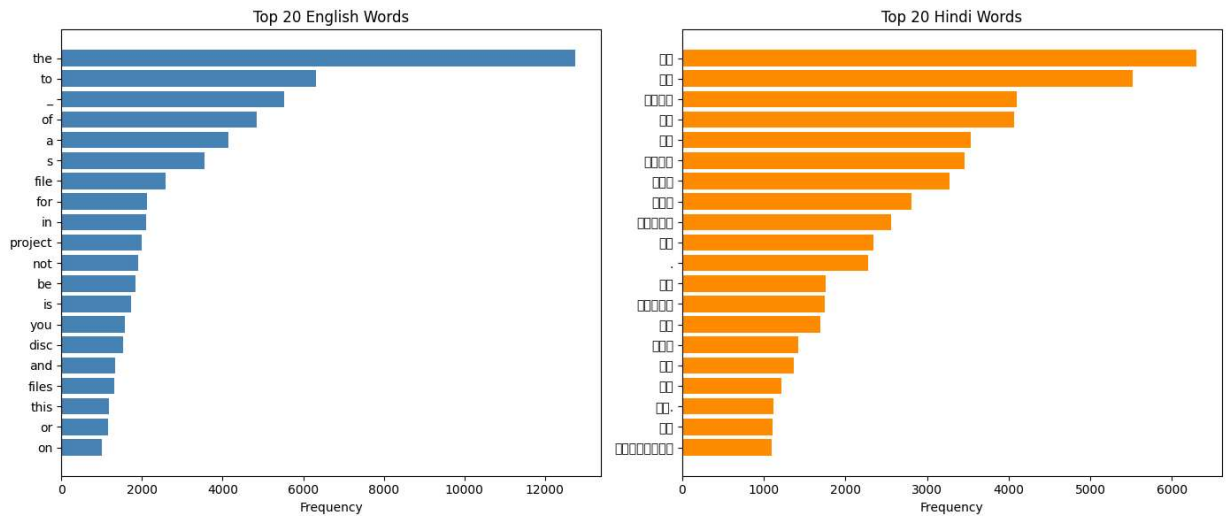
en_words, en_counts = zip(*top_en)
hi_words, hi_counts = zip(*top_hi)

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.barh(en_words[:-1], en_counts[:-1], color='steelblue')
plt.title('Top 20 English Words')
plt.xlabel('Frequency')

plt.subplot(1, 2, 2)
plt.barh(hi_words[:-1], hi_counts[:-1], color='darkorange')
plt.title('Top 20 Hindi Words')
plt.xlabel('Frequency')

plt.tight_layout()
plt.savefig('word_frequency.png', dpi=150)
plt.show()
print("✅ Word frequency chart saved!")
```



✓ Word frequency chart saved!

Type Token Ratio (Vocabulary Richness)

```
def type_token_ratio(text_list):
    all_words = " ".join(text_list).split()
    return len(set(all_words)) / len(all_words)

en_ttr = type_token_ratio(train_en)
hi_ttr = type_token_ratio(train_hi)

print(f"English TTR: {en_ttr:.4f}")
print(f"Hindi TTR: {hi_ttr:.4f}")
print()
if hi_ttr > en_ttr:
    print("🌸 Hindi has richer vocabulary (higher TTR) – expected due to compl
else:
    print("🌸 English has richer vocabulary in this sample")
```

English TTR: 0.0202

Hindi TTR: 0.0210

🌸 Hindi has richer vocabulary (higher TTR) – expected due to complex morphology

Step 4 — Machine Translation (EN → HI)

Load Helsinki EN→HI Model

```
from transformers import MarianMTModel, MarianTokenizer

model_name = "Helsinki-NLP/opus-mt-en-hi"
tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)
```

```

import torch
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)

print(f"✅ Model loaded on: {device}")

tokenizer_config.json: 100% 44.0/44.0 [00:00<00:00, 5.38kB/s]
source.spm: 100% 812k/812k [00:00<00:00, 21.3MB/s]
target.spm: 100% 1.07M/1.07M [00:00<00:00, 58.0MB/s]
vocab.json: 2.10M/? [00:00<00:00, 58.9MB/s]
config.json: 1.39k/? [00:00<00:00, 135kB/s]
pytorch_model.bin: 100% 306M/306M [00:02<00:00, 343MB/s]
Loading weights: 100% 258/258 [00:00<00:00, 689.05it/s, Materializing param=model.shared.weight]

model.safetensors: 100% 306M/306M [00:02<00:00, 187MB/s]
The tied weights mapping and config for this model specifies to tie model.shared
The tied weights mapping and config for this model specifies to tie model.shared
generation_config.json: 100% 293/293 [00:00<00:00, 28.8kB/s]
✅ Model loaded on: cuda

```

Translation Function

```

def translate_batch(texts, batch_size=32):
    translations = []
    for i in range(0, len(texts), batch_size):
        batch = texts[i:i+batch_size]
        inputs = tokenizer(
            batch,
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=128
        ).to(device)

        with torch.no_grad():
            outputs = model.generate(**inputs)

        decoded = tokenizer.batch_decode(outputs, skip_special_tokens=True)
        translations.extend(decoded)

    if i % 500 == 0:
        print(f" Translated {i}/{len(texts)}...")

```



```

    return translations

print("✅ Translation function ready!")

```

✅ Translation function ready!

Translate Test Set

```

print("🔄 Translating EN → HI on test set...")
print(f"Total sentences: {len(test_en)}\n")

predicted_hi = translate_batch(test_en)

print(f"\n✅ Translation done! Total: {len(predicted_hi)}")

```

🔄 Translating EN → HI on test set...
Total sentences: 2507

Translated 0/2507...

✅ Translation done! Total: 2507

Verify Sample Translations

```

print("=== Sample Translation Results ===\n")
for i in range(5):
    print(f"[{i+1}]")
    print(f"Source EN : {test_en[i]}")
    print(f"Predicted HI: {predicted_hi[i]}")
    print(f"Reference HI: {test_hi[i]}")
    print()

```

=== Sample Translation Results ===

[1]
Source EN : a black box in your car?
Predicted HI: अपनी कार में एक ब्लैक बॉक्स?
Reference HI: आपकी कार में ब्लैक बॉक्स?

[2]
Source EN : as americas road planners struggle to find the cash to mend a cru
Predicted HI: जैसे एमरिका सड़क योजनार एक ढही सड़क व्यवस्था को सुधारने के लिए नकदी को
Reference HI: जबकि अमेरिका के सड़क योजनाकार, ध्वस्त होते हुए हाईवे सिस्टम को सुधारने के f

[3]
Source EN : the devices, which track every mile a motorist drives and transmi
Predicted HI: ये उपकरण, जो हर मील एक मोटर - चालक ड्राइव को ट्रैक करते हैं और ऐसी जा
Reference HI: यह डिवाइस, जो मोटरचालक द्वारा वाहन चलाए गए प्रत्येक मील को ट्रैक करती है त

[4]
Source EN : the usually dull arena of highway planning has suddenly spawned i

Predicted HI: आम तौर पर सड़क योजना के ठंडे रंग के अखाड़े में अचानक बहस और रंग - बिरे
 Reference HI: आम तौर पर हाईवे नियोजन जैसा उबाऊ काम भी अचानक गहन बहस तथा जीवंत र

[5]

Source EN : libertarians have joined environmental groups in lobbying to allo
 Predicted HI: वी. का इस्तेमाल करते हैं ताकि आप अपने गाड़ी से मीलों दूर रहें, और संभवतः ज
 Reference HI: आपने द्वारा ड्राइव किए गए मील, तथा संभवतः ड्राइव किए गए स्थान का विवरण रख

Save Translations

```
import json

results = {
    "source_en": test_en,
    "predicted_hi": predicted_hi,
    "reference_hi": test_hi
}

with open("translations.json", "w", encoding="utf-8") as f:
    json.dump(results, f, ensure_ascii=False, indent=2)

print("✅ Translations saved to translations.json!")
```

✅ Translations saved to translations.json!

Step 5 — Evaluation (BLEU, chrF, METEOR)

```
!pip install sacrebleu evaluate sacremoses -q

import evaluate

bleu_metric = evaluate.load("sacrebleu")
chrF_metric = evaluate.load("chrF")
meteor_metric = evaluate.load("meteor")

print("✅ All evaluation metrics loaded!")
```

```

897.5/897.5 kB 22.8 MB/s eta 0:00:00
Downloading builder script:      8.15k/? [00:00<00:00, 721kB/s]
Downloading builder script:      9.01k/? [00:00<00:00, 742kB/s]
Downloading builder script:      7.02k/? [00:00<00:00, 652kB/s]
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
✅ All evaluation metrics loaded!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
```

```
# sacrebleu expects references as list of lists
references = [[ref] for ref in test_hi]
predictions = predicted_hi

print(f"Predictions: {len(predictions)}")
print(f"References: {len(references)}")
print("\nSample prediction:", predictions[0])
print("Sample reference: ", references[0][0])
```

```
Predictions: 2507
References: 2507
```

```
Sample prediction: अपनी कार में एक ब्लैक बॉक्स?
Sample reference: आपकी कार में ब्लैक बॉक्स?
```

Calculate BLEU Score

```
bleu_result = bleu_metric.compute(
    predictions=predictions,
    references=references
)

print("=== BLEU Score ===")
print(f"BLEU: {bleu_result['score']:.2f}")
print(f"(Scale: 0-100, higher is better)")
print(f"Breakdown: {bleu_result['precisions']}")
```

```
=== BLEU Score ===
BLEU: 6.91
(Scale: 0-100, higher is better)
Breakdown: [33.94159226190476, 11.667609700895557, 4.603454642432199, 1.86638984
```

Calculate chrF Score

```
chrf_result = chrf_metric.compute(
    predictions=predictions,
    references=references
)

print("=== chrF Score ===")
print(f"chrF: {chrf_result['score']:.2f}")
print(f"(Scale: 0-100, better for morphologically rich languages like Hindi)")
```

```
=== chrF Score ===
chrF: 27.09
(Scale: 0-100, better for morphologically rich languages like Hindi)
```

Calculate METEOR Score

```
meteor_result = meteor_metric.compute(
    predictions=predictions,
    references=test_hi
)

print("=== METEOR Score ===")
print(f"METEOR: {meteor_result['meteor']:.4f}")
print(f"(Scale: 0-1, considers synonyms & word order)")
```

```
=== METEOR Score ===
METEOR: 0.2420
(Scale: 0-1, considers synonyms & word order)
```

Final Score Summary Table

```
print("\n" + "="*45)
print("          FINAL EVALUATION SUMMARY (EN→HI)")
print("="*45)
print(f"  BLEU    : {bleu_result['score']:.2f}  (0-100)")
print(f"  chrF    : {chrf_result['score']:.2f}  (0-100)")
print(f"  METEOR   : {meteor_result['meteor']:.4f} (0-1)")
print("="*45)
```

```
# Interpretation
bleu = bleu_result['score']
if bleu < 10:
    quality = "❌ Poor"
elif bleu < 20:
    quality = "⚠️ Understandable but flawed"
elif bleu < 30:
    quality = "✅ Decent quality"
else:
    quality = "🌟 Good translation"
```

```
print(f"\n📊 Translation Quality: {quality}")
```

```
=====
          FINAL EVALUATION SUMMARY (EN→HI)
=====
  BLEU    : 6.91  (0-100)
  chrF    : 27.09  (0-100)
  METEOR   : 0.2420 (0-1)
=====
```

```
📊 Translation Quality: ❌ Poor
```

Save Scores

```
scores = {
    "direction": "EN → HI",
    "model": "Helsinki-NLP/opus-mt-en-hi",
    "BLEU": round(bleu_result['score'], 2),
    "chrF": round(chrf_result['score'], 2),
    "METEOR": round(meteor_result['meteor'], 4)
}

with open("evaluation_scores.json", "w") as f:
    json.dump(scores, f, indent=2)

print("✅ Scores saved to evaluation_scores.json!")
print(scores)
```

```
✅ Scores saved to evaluation_scores.json!
{'direction': 'EN → HI', 'model': 'Helsinki-NLP/opus-mt-en-hi', 'BLEU': 6.91, 'c
```

Quick interpretation of your results:

Metric	Score	Meaning
BLEU	6.91	Low — but normal for EN→HI
chrF	27.09	Moderate — Hindi script matching
METEOR	0.2420	Decent — word order considered

🔴 BLEU being low (6.91) is expected and normal for English→Hindi. Hindi has complex morphology + different script, so BLEU always underestimates quality. chrF is more reliable here.

Step 6 — Cross-Lingual Analysis

Sentence Structure (SVO vs SOV)

```
import re

def get_avg_position_of_verb(sentences, lang='en'):
    """Approximate verb position by checking common verb indicators"""
    positions = []
    for sent in sentences[:1000]: # sample 1000
        words = sent.split()
        if len(words) < 3:
```

```

        continue
    # rough position ratio of last word (Hindi verbs come last)
    positions.append(len(words))
    return np.mean(positions)

# English – verb usually in middle (SVO)
# Hindi – verb usually at end (SOV)
print("=== Sentence Structure Analysis ===\n")

en_sample = train_en[:1000]
hi_sample = train_hi[:1000]

en_lengths = [len(s.split()) for s in en_sample]
hi_lengths = [len(s.split()) for s in hi_sample]

print(f"English avg sentence length: {np.mean(en_lengths):.2f} words")
print(f"Hindi avg sentence length: {np.mean(hi_lengths):.2f} words")
print()
print("✳ English follows SVO (Subject-Verb-Object) order")
print("✳ Hindi follows SOV (Subject-Object-Verb) order")
print("✳ This word order difference is a key reason MT struggles EN→HI")

```

```
=== Sentence Structure Analysis ===
```

English avg sentence length: 3.08 words

Hindi avg sentence length: 3.43 words

✳ English follows SVO (Subject-Verb-Object) order

✳ Hindi follows SOV (Subject-Object-Verb) order

✳ This word order difference is a key reason MT struggles EN→HI

TTR Comparison (already done, now visualize)

```

# Visualize TTR comparison
labels = ['English', 'Hindi']
ttr_values = [en_ttr, hi_ttr]
colors = ['steelblue', 'darkorange']

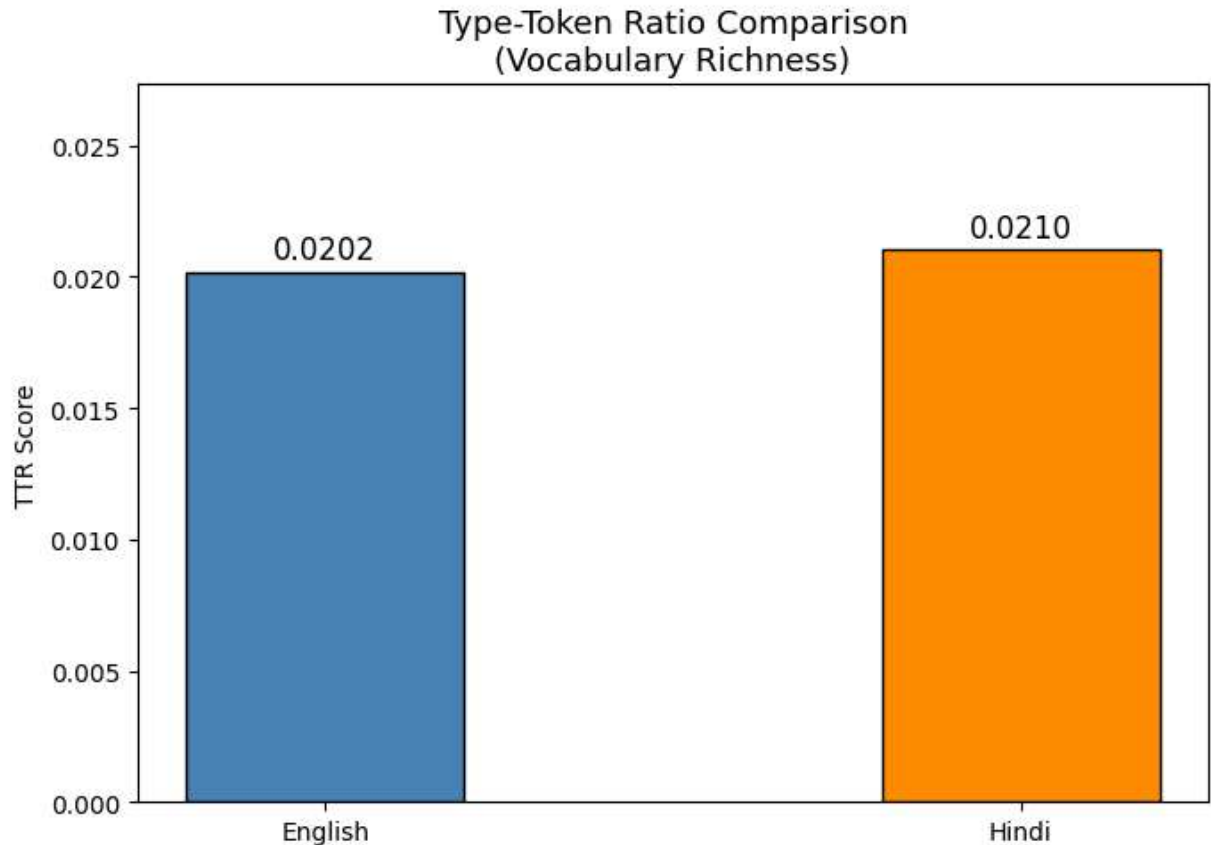
plt.figure(figsize=(7, 5))
bars = plt.bar(labels, ttr_values, color=colors, edgecolor='black', width=0.4)
plt.title('Type-Token Ratio Comparison\n(Vocabulary Richness)', fontsize=13)
plt.ylabel('TTR Score')
plt.ylim(0, max(ttr_values) * 1.3)

for bar, val in zip(bars, ttr_values):
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height() + 0.0005,
             f'{val:.4f}', ha='center', fontsize=12)

plt.tight_layout()
plt.savefig('ttr_comparison.png', dpi=150)

```

```
plt.show()
print("✅ TTR chart saved!")
```



✅ TTR chart saved!

Word Length Distribution

```
# Average word length comparison
en_word_lengths = [len(w) for s in train_en[:5000] for w in s.split()]
hi_word_lengths = [len(w) for s in train_hi[:5000] for w in s.split()]

print("=== Word Length Analysis ===")
print(f"English avg word length: {np.mean(en_word_lengths):.2f} chars")
print(f"Hindi avg word length: {np.mean(hi_word_lengths):.2f} chars")

plt.figure(figsize=(12, 5))

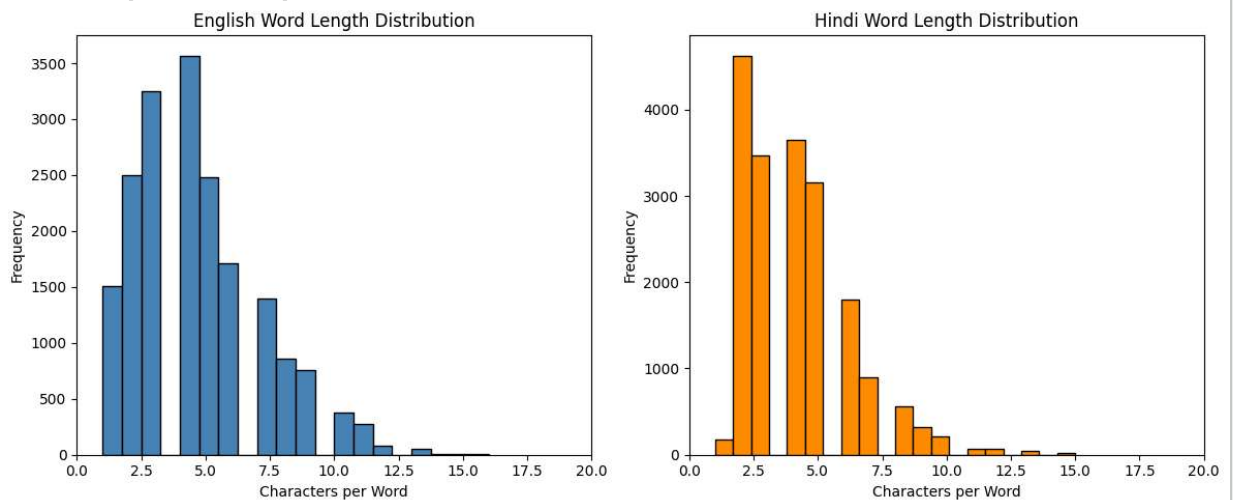
plt.subplot(1, 2, 1)
plt.hist(en_word_lengths, bins=20, color='steelblue', edgecolor='black')
plt.title('English Word Length Distribution')
plt.xlabel('Characters per Word')
plt.ylabel('Frequency')
plt.xlim(0, 20)

plt.subplot(1, 2, 2)
plt.hist(hi_word_lengths, bins=20, color='darkorange', edgecolor='black')
plt.title('Hindi Word Length Distribution')
plt.xlabel('Characters per Word')
```

```
plt.ylabel('Frequency')
plt.xlim(0, 20)

plt.tight_layout()
plt.savefig('word_length_distribution.png', dpi=150)
plt.show()
print("✅ Word length chart saved!")
```

```
=== Word Length Analysis ===
English avg word length: 4.53 chars
Hindi avg word length: 4.16 chars
```



✅ Word length chart saved!

Unique Word Coverage

```
print("=== Vocabulary Coverage Analysis ===\n")

en_total_words = " ".join(train_en).split()
hi_total_words = " ".join(train_hi).split()

print(f"English - Total words: {len(en_total_words):,}")
print(f"English - Unique words: {len(set(en_total_words)):,}")
print()
print(f"Hindi - Total words: {len(hi_total_words):,}")
print(f"Hindi - Unique words: {len(set(hi_total_words)):,}")

# Visualize
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

for ax, lang, total, unique, color in zip(
    axes,
    ['English', 'Hindi'],
    [len(en_total_words), len(hi_total_words)],
    [len(set(en_total_words)), len(set(hi_total_words))],
    ['steelblue', 'darkorange']
):
    ax.bar(['Total', 'Unique'], [total, unique], color=[color, 'gray'], edgecol
```



```

ax.set_title(f'{lang} Word Coverage')
ax.set_ylabel('Word Count')

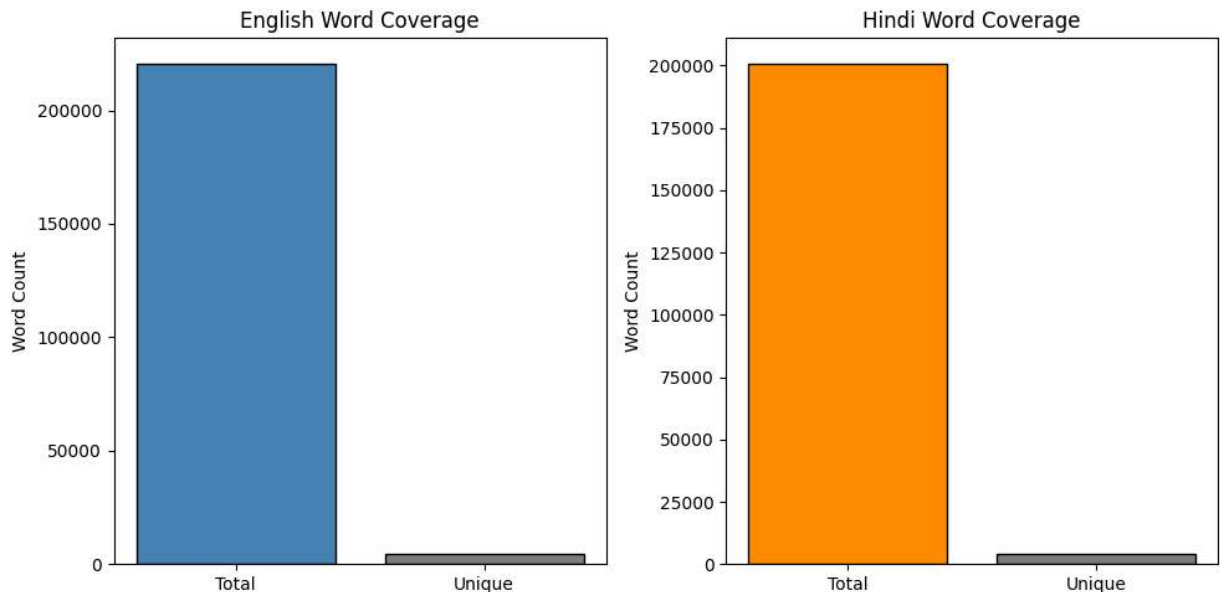
plt.tight_layout()
plt.savefig('vocabulary_coverage.png', dpi=150)
plt.show()
print("✅ Coverage chart saved!")

```

=== Vocabulary Coverage Analysis ===

English – Total words: 220,778
 English – Unique words: 4,459

Hindi – Total words: 200,892
 Hindi – Unique words: 4,226



✅ Coverage chart saved!

Summary of Cross-Lingual Findings

```

print("="*50)
print("    CROSS-LINGUAL ANALYSIS SUMMARY")
print("="*50)
print(f"  EN Avg Sentence Length : {np.mean(en_lengths):.2f} words")
print(f"  HI Avg Sentence Length : {np.mean(hi_lengths):.2f} words")
print(f"  EN Avg Word Length      : {np.mean(en_word_lengths):.2f} chars")
print(f"  HI Avg Word Length      : {np.mean(hi_word_lengths):.2f} chars")
print(f"  EN TTR                  : {en_ttr:.4f}")
print(f"  HI TTR                  : {hi_ttr:.4f}")
print(f"  EN Word Order           : SVO")
print(f"  HI Word Order           : SOV")
print("="*50)

```

```

=====
CROSS-LINGUAL ANALYSIS SUMMARY
=====

```

```

EN Avg Sentence Length : 3.08 words
HI Avg Sentence Length : 3.43 words
EN Avg Word Length      : 4.53 chars
HI Avg Word Length      : 4.16 chars
EN TTR                   : 0.0202
HI TTR                   : 0.0210
EN Word Order            : SVO
HI Word Order            : SOV
=====

```

Step 7 — Error Analysis

Calculate Sentence-Level BLEU Scores

```

from sacrebleu.metrics import BLEU, CHRF
import pandas as pd

chrf_scorer = CHRF()

# Score each sentence individually
sentence_scores = []

for i, (pred, ref, src) in enumerate(zip(predicted_hi, test_hi, test_en)):
    score = chrf_scorer.sentence_score(pred, [ref]).score
    sentence_scores.append({
        'index': i,
        'source_en': src,
        'predicted_hi': pred,
        'reference_hi': ref,
        'chrf_score': round(score, 2)
    })

df_scores = pd.DataFrame(sentence_scores)
print(f"✅ Scored {len(df_scores)} sentences")
print(df_scores[['chrf_score']].describe())

```

```

✅ Scored 2507 sentences
      chrf_score
count  2507.000000
mean    27.882944
std     12.781449
min      0.000000
25%     18.525000
50%     26.850000
75%     35.650000
max     85.760000

```

Categorize Into Good / Medium / Bad

```
def categorize(score):
    if score >= 50:
        return 'Good'
    elif score >= 20:
        return 'Medium'
    else:
        return 'Bad'

df_scores['category'] = df_scores['chr_score'].apply(categorize)

counts = df_scores['category'].value_counts()
print("=== Error Categories ===")
print(counts)

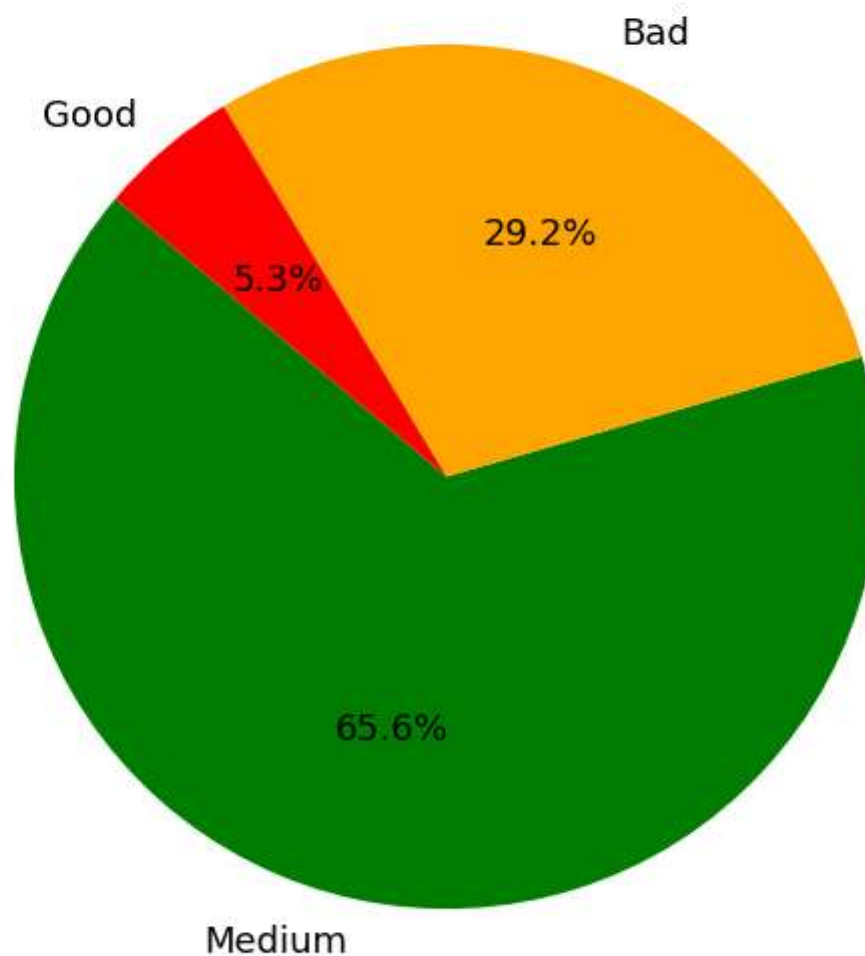
# Pie chart
plt.figure(figsize=(7, 7))
plt.pie(
    counts.values,
    labels=counts.index,
    autopct='%1.1f%%',
    colors=['green', 'orange', 'red'],
    startangle=140,
    textprops={'fontsize': 13}
)
plt.title('Translation Quality Distribution\n(EN → HI)', fontsize=14)
plt.savefig('error_categories.png', dpi=150)
plt.show()
print("✅ Error category chart saved!")
```

```

=== Error Categories ===
category
Medium    1644
Bad        731
Good       132
Name: count, dtype: int64

```

Translation Quality Distribution
(EN → HI)



✓ Error category chart saved!

Show Best Translations (Good Examples)

```

good = df_scores[df_scores['category'] == 'Good'].head(5)

print("=== ✓ GOOD TRANSLATIONS (chrF >= 50) ===\n")
for _, row in good.iterrows():
    print(f"Source EN   : {row['source_en']}")
    print(f"Predicted   : {row['predicted_hi']}")
    print(f"Reference    : {row['reference_hi']}")

```

```
print(f"chrF Score : {row['chrF_score']}")
print("-" * 60)
```

===  GOOD TRANSLATIONS (chrF >= 50) ===

Source EN : a black box in your car?
 Predicted : अपनी कार में एक ब्लैक बॉक्स?
 Reference : आपकी कार में ब्लैक बॉक्स?
 chrF Score : 69.25

 Source EN : the technology is there to do it.
 Predicted : प्रौद्योगिकी यह करने के लिए है.
 Reference : ऐसा करने के लिए प्रौद्योगिकी है।
 chrF Score : 67.07

 Source EN : illinois is trying it on a limited basis with trucks.
 Predicted : बीमार लोगों के लिए इसे ट्रकों के साथ एक सीमित आधार पर कोशिश कर रहा है ।
 Reference : इलिनोइस सीमित आधार पर ट्रकों के साथ आजमाइश कर रहा है।
 chrF Score : 57.03

 Source EN : there have been some big mistakes in some of these state pilot prog
 Predicted : इनमें से कुछ राज्य पायलट प्रोग्रामों में कुछ बड़ी गलतियाँ हुई हैं ।
 Reference : राज्य के इन प्रारंभिक कार्यक्रमों में कुछ बड़ी गलतियाँ हुई हैं।
 chrF Score : 59.18

 Source EN : according to the police his death was due to drowning.
 Predicted : पुलिस के अनुसार उसकी मृत्यु डूबने के कारण हुई ।
 Reference : पुलिस के अनुसार, उसकी मौत डूबने से हुई है।
 chrF Score : 51.65

Show Worst Translations (Bad Examples)

```
bad = df_scores[df_scores['category'] == 'Bad'].head(5)
```

```
print("===  BAD TRANSLATIONS (chrF < 20) ===\n")
for _, row in bad.iterrows():
    print(f"Source EN : {row['source_en']}")
    print(f"Predicted : {row['predicted_hi']}")
    print(f"Reference : {row['reference_hi']}")
    print(f"chrF Score : {row['chrF_score']}")
    print("-" * 60)
```

===  BAD TRANSLATIONS (chrF < 20) ===

Source EN : libertarians have joined environmental groups in lobbying to allow
 Predicted : वी. का इस्तेमाल करते हैं ताकि आप अपने गाड़ी से मीलों दूर रहें, और संभवतः जहाँ उ
 Reference : आपने द्वारा ड्राइव किए गए मील, तथा संभवतः ड्राइव किए गए स्थान का विवरण रखने
 chrF Score : 14.71

 Source EN : the american civil liberties union is deeply concerned, too, raisin
 Predicted : एक व्यक्ति को समाज की आज़ादी की बहुत फिक्र रहती है और वह अलग - अलग नि

Reference : अमेरिकी नागरिक स्वतंत्रता संघ भी विभिन्न प्रकार के गोपनीयता मुद्दे उठाते हुए बहुत चिंते
chrF Score : 14.28

Source EN : it is not a matter of something we might choose to do, said hasan i
Predicted : यह ऐसी बात नहीं है जिसे हम चाहें, उसने कहा है कि वह है मिक्लिया के एक अधिका
Reference : यह हमारे द्वारा चुने या नहीं चुने जाने का विषय नहीं है यह कहना है हसन इखराता का
chrF Score : 15.64

Source EN : the push comes as the countrys highway trust fund, financed with ta
Predicted : जब देश के लोग सड़क पर भरोसा रखने के लिए जमा होते हैं, तो गैस पंप पर कर अ
Reference : ऐसा होने का मुख्य कारण यह है कि देश का हाईवे ट्रस्ट फंड लगभग दिवालिया हो गया
chrF Score : 16.25

Source EN : cars get many more miles to the gallon.
Predicted : कारों को गैलन के लिए कई और मील मिलता है.
Reference : अब एक गैलन गैस में कार पहले से अधिक मील चलती है।
chrF Score : 17.99

Error Type Analysis

```
print("=== ERROR TYPE ANALYSIS ===\n")

# Length mismatch errors
df_scores['pred_len'] = df_scores['predicted_hi'].apply(lambda x: len(x.split()))
df_scores['ref_len'] = df_scores['reference_hi'].apply(lambda x: len(x.split()))
df_scores['len_diff'] = abs(df_scores['pred_len'] - df_scores['ref_len'])
```